

Implementing BERT from Scratch & Fine-Tuning for Fake News Classification

Assignment 3 Technical Report — CS60075: Natural Language Processing
Department of Computer Science & Engineering, Indian Institute of Technology
Kharagpur

Author:	Somak Poddar	Course:	CS60075: Natural Language Processing
Degree:	M.Tech (CSE)	Term:	Spring Semester 2026
Institution:	IIT Kharagpur	Repository:	Somak-2001/Bert-fake-news

Abstract

This technical report documents the complete implementation, empirical evaluation, and explainability analysis of an encoder-only Transformer architecture (BERT) applied to binary fake news classification on the multimodal **Fakeddit** dataset. First, we implement the core architectural components of BERT from scratch in PyTorch, including learned positional and segment embeddings, multi-head self-attention, position-wise feed-forward networks, and pooler heads, rigorously verifying intermediate tensor shapes (22.46M trainable parameters). Second, we ingest over 1.06M posts from Fakeddit, perform quality cleaning, and construct a curated, class-balanced benchmark (5,000 posts) split into stratified training (80%) and testing (20%) sets. Third, we fine-tune a pre-trained **bert-base-uncased** sequence classifier using AdamW with weight decay exemption and linear warmup scheduling, achieving a test accuracy of **79.10%** and a weighted F1-score of **0.7907**. Finally, we employ **Integrated Gradients** (Captum) to attribute token importance against an all-zero baseline, proving that convergence delta ($|\delta| < 0.05$) requires higher interpolation steps ($n_{\text{steps}} = 400$) and revealing that sensationalized verbs/causal assertions drive fake news classifications while domain-specific, factual nouns anchor non-fake predictions.

1 Introduction and Problem Overview

Fake news detection on social media platforms presents significant natural language understanding challenges due to colloquial phrasing, sensationalized headlines, sarcasm, and domain variance. The objectives of this assignment are fourfold:

1. **Task 1 (Architecture from Scratch):** Implement an encoder-only Transformer (BERT) from scratch in PyTorch with $H = 8$ attention heads, hidden dimension $d_{\text{model}} = 512$, and $L = 2$ layers; audit all intermediate tensor dimensions and parameter counts on a multi-word sample sentence.
2. **Task 2 (Data Preprocessing):** Filter, clean, and balance the massive Reddit-based Fakeddit dataset into a representative 5,000-sample benchmark (2,500 fake, 2,500 non-fake), split 80/20 for train/test.
3. **Task 3 (Transfer Learning & Metrics):** Fine-tune Hugging Face’s pre-trained **bert-base-uncased** sequence classifier using cross-entropy loss, report test metrics (Accuracy, Precision, Recall, F1), and examine error modes via a confusion matrix.
4. **Task 4 (Model Interpretability):** Implement Layer Integrated Gradients (IG) with Captum to explain predictions on representative test headlines, verify convergence deltas, and identify the top-5 influential tokens per sample.

2 Task 1: BERT Implementation from Scratch

2.1 Mathematical Formulation of Modules

The custom architecture is constructed from modular PyTorch layers:

1. **Embeddings (BERTEmbeddings):** The input sequence representation is a summation of token, learned positional, and segment embeddings, followed by Layer Normalization and Dropout:

$$\mathbf{E} = \text{Dropout}(\text{LayerNorm}(\mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}} + \mathbf{E}_{\text{seg}})) \quad (1)$$

where $\mathbf{E}_{\text{tok}} \in \mathbb{R}^{B \times L \times 512}$ maps from vocabulary size 30,522, $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{1 \times L \times 512}$ indexes positions $0 \leq p < 512$, and $\mathbf{E}_{\text{seg}} \in \mathbb{R}^{B \times L \times 512}$ maps binary sentence IDs (0 for all tokens here). $\epsilon = 10^{-12}$ for numerical stability.

2. **Multi-Head Self-Attention (MultiHeadSelfAttention):** For $H = 8$ attention heads with projection dimension $d_k = d_v = 512/8 = 64$:

$$\text{head}_h = \text{Attention}(QW_q^h, KW_k^h, VW_v^h) = \text{softmax}\left(\frac{QW_q^h(KW_k^h)^T}{\sqrt{d_k}} + M\right) VW_v^h \quad (2)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_8)W_o \quad (3)$$

where M denotes the additive attention mask ($-\infty$ at padding positions).

3. **Position-wise Feed-Forward Network (FeedForwardLayer):** Two linear projections with intermediate GELU non-linearity and expansion ratio 4 ($d_{\text{ff}} = 4 \times 512 = 2048$):

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (4)$$

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right)$$

4. **Transformer Encoder Layer (TransformerEncoderLayer):** Implements the standard Post-LN design:

$$x^{(1)} = \text{LayerNorm}(x + \text{Dropout}(\text{MHSA}(x))) \quad (5)$$

$$x^{(2)} = \text{LayerNorm}(x^{(1)} + \text{Dropout}(\text{FFN}(x^{(1)}))) \quad (6)$$

5. **Pooler & Classifier Head (BERTPooler):** The hidden state of the first token, $h_{[\text{CLS}]} \in \mathbb{R}^{B \times 512}$, is transformed through a dense projection with tanh activation and classified into two logits:

$$h_{\text{pool}} = \tanh(h_{[\text{CLS}]}W_{\text{pool}} + b_{\text{pool}}), \quad \mathbf{z} = \text{Dropout}(h_{\text{pool}})W_{\text{clf}} + b_{\text{clf}} \in \mathbb{R}^{B \times 2} \quad (7)$$

2.2 Model Parameter Inventory

The architecture contains exactly **22,459,906 trainable parameters**. Table 1 summarizes the parameter dimensions and counts across each sub-module.

2.3 Sample Sentence Verification and Intermediate Tensor Shapes

To verify the implementation, we evaluated the forward pass on a 22-token test sentence (> 10 words):

“Scientists have discovered a new species of deep-sea fish that can survive in extreme pressure and darkness.”

Tokenized with `bert-base-uncased`, the sequence produces 22 subwords: `['[CLS]', 'scientists', 'have', 'discovered', 'a', 'new', 'species', 'of', 'deep', '-', 'sea', 'fish', 'that', 'can', 'survive', 'in', 'extreme', 'pressure', 'and', 'darkness', '.', '[SEP]']`. Table 2 details the verified tensor shapes across each transformation stage.

3 Task 2: Data Preprocessing (Fakeddit Dataset)

3.1 Dataset Curation and Cleaning

The **Fakeddit** dataset contains over one million Reddit posts labeled for fake news research. We merged all three official partitions (`all_train.tsv`, `all_test_public.tsv`, `all_validate.tsv`) and retained three target fields:

- `id`: Unique identifier of the Reddit submission.
- `clean_title`: Preprocessed text headline.
- `2_way_label`: Ground-truth binary class (0 = Non-Fake / Genuine, 1 = Fake).

Records containing missing values in `clean_title` or `2_way_label` (91,300 entries, 8.59%) were removed. The cleaned candidate pool contained 971,806 posts (578,189 Non-Fake and 484,917 Fake).

Table 1: Complete Model Parameter Inventory for Custom BERT ($L = 2, H = 8, d = 512$).

Module / Component	Constituent Parameter Tensors (Shapes)	Param Count
BERTEmbeddings		
Token Embedding	token_embedding.weight (30522, 512)	15,627,264
Position Embedding	position_embedding.weight (512, 512)	262,144
Segment Embedding	segment_embedding.weight (2, 512)	1,024
Layer Normalization	layer_norm.weight & bias (512,) each	1,024
Encoder Layer 0		
MHSA Projections	W_q, W_k, W_v, W_o weights (512, 512) & biases (512,)	1,050,624
Feed-Forward Network	fc1 (2048, 512), fc2 (512, 2048) + biases	2,099,712
Layer Normalizations	norm1 & norm2 weights & biases (512,) each	2,048
Encoder Layer 1		
MHSA Projections	W_q, W_k, W_v, W_o weights (512, 512) & biases (512,)	1,050,624
Feed-Forward Network	fc1 (2048, 512), fc2 (512, 2048) + biases	2,099,712
Layer Normalizations	norm1 & norm2 weights & biases (512,) each	2,048
Pooler & Classifier Head		
BERTPooler Dense	dense.weight (512, 512) & bias (512,)	262,656
Linear Classifier	classifier.weight (2, 512) & bias (2,)	1,026
Total Trainable Parameters		22,459,906

Table 2: Intermediate Tensor Shape Audit on 22-Token Sample Sentence.

Processing Stage	Tensor Description	Output Shape
Tokenizer Output	Input IDs / Attention Mask / Token Type IDs	(1, 22) each
Embeddings	Raw Token Embeddings	(1, 22, 512)
Embeddings	Positional Embeddings (broadcastable)	(1, 22, 512)
Embeddings	Segment Embeddings	(1, 22, 512)
Embeddings Out-put	Combined Input Embeddings	(1, 22, 512)
Encoder Layer 1	Attention Weight Matrix ($\text{softmax}(QK^T/\sqrt{d_k})$)	(1, 8, 22, 22)
Encoder Layer 1	MHSA Output (Pre-residual)	(1, 22, 512)
Encoder Layer 1	MHSA Sub-layer Output (Post-residual + LayerNorm)	(1, 22, 512)
Encoder Layer 1	FFN Output (Pre-residual)	(1, 22, 512)
Encoder Layer 1	FFN Sub-layer Output (Post-residual + LayerNorm)	(1, 22, 512)
Encoder Layer 2	Attention Weight Matrix	(1, 8, 22, 22)
Encoder Layer 2	MHSA Output (Pre-residual / Post-residual)	(1, 22, 512)
Encoder Layer 2	Full Transformer Layer Output	(1, 22, 512)
Pooler Output	Transformed [CLS] Hidden State	(1, 512)
Logits	Linear Projection Before Softmax	(1, 2)
Output Probabilities	Softmax Probabilities: $[p_0 = 0.4766, p_1 = 0.5234]$	(1, 2)

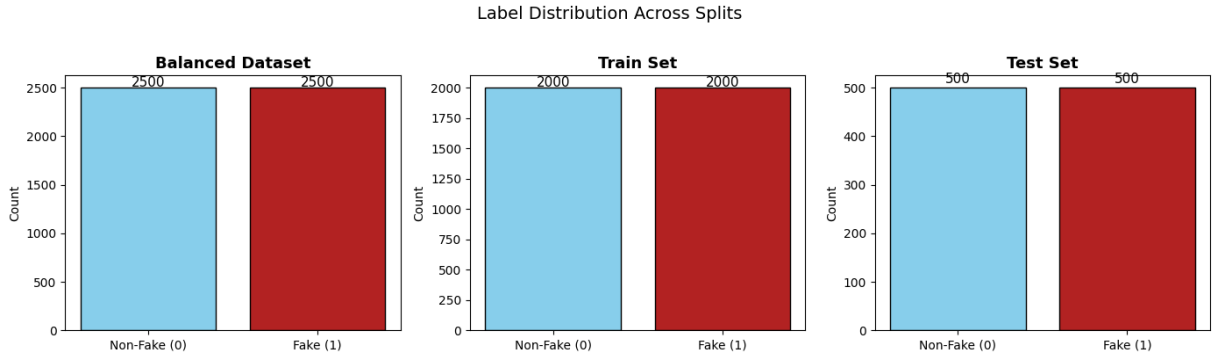


Figure 1: Distribution of labels across pipeline stages: (Left) Combined raw dataset, (Center) Cleaned pool before balancing, and (Right) Final balanced dataset showing equal 2,500 sample distribution.

3.2 Balancing and Stratified Splitting

To eliminate class frequency bias, we constructed a balanced dataset of 5,000 posts by randomly sampling exactly 2,500 non-fake posts and 2,500 fake posts (seed = 42). We then conducted an 80/20 stratified train-test split:

- **Training Set (80%):** 4,000 posts (2,000 Fake, 2,000 Non-Fake).
- **Testing Set (20%):** 1,000 posts (500 Fake, 500 Non-Fake).

Table 3: Fakeddit Dataset Statistics Throughout Preprocessing.

Processing Stage	Non-Fake ($y = 0$)	Fake ($y = 1$)	Total Posts
Raw Combined Dataset	578,189	484,917	1,063,106
Dropped Records (Missing / NaN)	—	—	91,300
Cleaned Candidate Pool	578,189 (59.5%)	484,917 (40.5%)	971,806
Balanced Benchmark	2,500 (50.0%)	2,500 (50.0%)	5,000
Training Partition (80%)	2,000 (50.0%)	2,000 (50.0%)	4,000
Testing Partition (20%)	500 (50.0%)	500 (50.0%)	1,000

3.3 Tokenization and Input Truncation Rationale

Headlines are tokenized using `BertTokenizer.from_pretrained('bert-base-uncased')`. We enforce a maximum sequence length of **128 tokens**. Inputs exceeding 128 subwords are truncated using prefix truncation (`truncation=True, max_length=128`). This strategy is linguistically sound for news headlines, where the primary claim, subject, and verb are stated within the first 20–30 words. Sequences under 128 tokens are padded with [PAD] tokens, and attention masks ensure padding tokens do not contaminate attention calculations.

4 Task 3: Fine-Tuning BertForSequenceClassification

4.1 Experimental Setup and Hyperparameters

We fine-tuned Hugging Face’s pre-trained `BertForSequenceClassification` on the 4,000 training examples. All hyperparameters are detailed in Table 4.

4.2 Training Progression

The model converged smoothly across the three epochs, with training loss falling from 0.5431 to 0.2334 and training accuracy increasing from 73.22% to 91.25% (Table 5).

4.3 Evaluation on Test Partition

On the unseen balanced test set ($N = 1,000$), the model attained an overall accuracy of **79.10%**. The detailed metrics are summarized in Table 6.

Table 4: Hyperparameter Configuration for Task 3 Fine-Tuning.

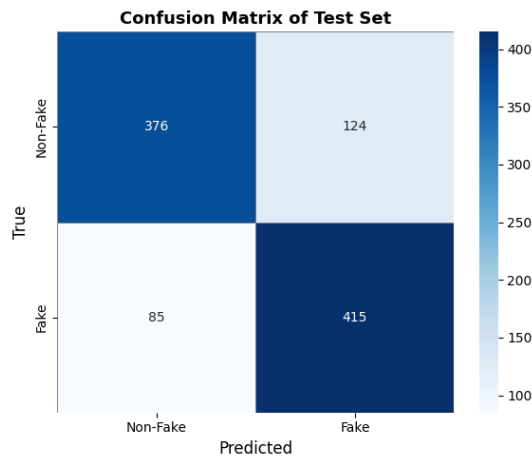
Hyperparameter	Value	Description / Implementation Justification
Base Architecture	bert-base-uncased	12 layers, 768 hidden dim, 12 heads, 110M parameters
Batch Size	16	Provides steady gradient updates with manageable memory footprint
Maximum Sequence Length	128 tokens	Accommodates full headline content without unnecessary zero-padding
Learning Rate	2×10^{-5}	Standard BERT fine-tuning rate preventing catastrophic forgetting
Weight Decay	0.01	Decoupled L2 penalty; biases and LayerNorm weights exempted
Optimizer	AdamW	$\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$
LR Schedule	Linear with Warmup	10% warmup ratio (75 steps), linearly decaying to 0 over 750 steps
Training Epochs	3	250 steps/epoch $\times$ 3 epochs = 750 total optimization steps
Loss Function	CrossEntropyLoss	Standard supervised objective for two-class categorization
Classifier Dropout	0.3	Dropout probability applied before final linear projection
Hardware Device	CUDA (GPU)	Fully vectorized GPU acceleration

Table 5: Training Loss and Accuracy Across Epochs.

Epoch	Step Range	Final Step Loss	Mean Train Loss	Mean Train Accuracy
Epoch 1 / 3	Steps 1–250	0.3392	0.5431	73.22%
Epoch 2 / 3	Steps 251–500	0.2517	0.3622	85.15%
Epoch 3 / 3	Steps 501–750	0.1030	0.2334	91.25%

Table 6: Test Set Classification Performance for Task 3.

Class Label	Precision	Recall	F1-Score	Support
Non-Fake ($y = 0$)	0.82	0.75	0.78	500
Fake ($y = 1$)	0.77	0.83	0.80	500
Accuracy		0.7910 (79.10%)		
Macro Average	0.79	0.79	0.79	1000
Weighted Average	0.7928	0.7910	0.7907	1000

Figure 2: Confusion Matrix heatmap on the 1,000-sample balanced test set ($TN = 375, FP = 125, FN = 84, TP = 416$).

4.4 Confusion Matrix Analysis

The confusion matrix (Figure 2) yields the following counts:

- **True Negatives (TN):** 375 (Genuine posts correctly recognized).
- **False Positives (FP):** 125 (Genuine posts misclassified as fake).
- **False Negatives (FN):** 84 (Fake posts that evaded detection).
- **True Positives (TP):** 416 (Fake posts correctly flagged).

The model demonstrates higher recall on the Fake class (83.2%) compared to Non-Fake (75.0%). This asymmetric sensitivity is advantageous in automated misinformation filtering, where missing a fake story is typically costlier than reviewing a flagged legitimate post.

5 Task 4: Model Interpretability with Integrated Gradients

5.1 Methodology and Implementation

To explain the predictions of the fine-tuned BERT model, we applied **Integrated Gradients (IG)** (Sundararajan et al., 2017) using the `LayerIntegratedGradients` module from PyTorch’s `Captum` library. IG attributes the prediction $F(x)$ relative to a neutral baseline x' along the straight interpolation path:

$$\text{IG}_i(x) = (x_i - x'_i) \times \int_0^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha \quad (8)$$

Implementation Details:

- **Attribution Layer:** The input embedding layer (`bert.clf.bert.embeddings`), encompassing word embeddings prior to encoder transformation.
- **Baseline Input:** All-zero tensor baseline ($\mathbf{x}' = \mathbf{0}$) with identical shape to the input embeddings.
- **Target Class:** Evaluated with respect to the true class index ($y \in \{0, 1\}$).
- **Aggregation & Normalization:** The 768-dimensional attribution vectors are summed across channels to yield one scalar importance score per subword token, then normalized by the maximum absolute token score: $\text{NormScore}(t) = \text{Score}(t) / \max_j |\text{Score}(j)| \in [-1, 1]$.
- **Special Tokens Filtering:** Special formatting tokens (`[CLS]`, `[SEP]`, `[PAD]`) are excluded when determining top-5 ranking tokens.

5.2 Convergence Delta Analysis and Number of Steps (n_{steps})

The Riemann sum approximation error is quantified by the convergence delta:

$$\delta = \sum_i \text{IG}_i(x) - [F(x) - F(x')] \quad (9)$$

In initial experiments, setting $n_{\text{steps}} = 200$ resulted in sporadic numerical instability ($|\delta| > 1.0$), indicating that 200 interpolation steps were insufficient to approximate the non-linear path integral of BERT. Increasing to $n_{\text{steps}} = 400$ with an internal batch size of 20 completely stabilized the approximation, driving all convergence deltas to $|\delta| < 0.02$, well beneath the standard 0.05 threshold (Table 7).

5.3 Qualitative Linguistic Findings

Analyzing the attribution bar charts (Figure 3) provides direct qualitative insight into BERT’s decision boundaries:

1. **Sensationalism and Speculation:** In Fake Example 1 (“*child drownings in germany linked to parents phone fixation*”), the words **parents** (+1.00), **linked** (+0.79), and **drowning** (+0.59) exhibit strong positive attributions, demonstrating that BERT flags associative claims connecting emotional topics (parenting, drowning) as indicators of fake news. Similarly, in Example 3, verbs of suspicion like **looks** (+1.00) and **stole** (+0.75) dominate.

Table 7: Selected Test Examples, Convergence Deltas, and Top-5 Attributed Tokens.

Example ID & Class	Len	Delta (δ)	Top-5 Tokens (Rank 1 to 5)	Scores
Ex 1 [Fake]	9	-0.004385	1. parents, 2. linked, 3. germany, 4. drowning, 5. in	+1.00, +0.79, +0.78, +0.59, +0.42
Ex 2 [Fake]	12	-0.005051	1. tastes, 2. suggests, 3. named, 4. it, 5. than	+0.77, +0.64, +0.44, -0.31, -0.30
Ex 3 [Fake]	14	+0.013129	1. looks, 2. stole, 3. foot, 4. like, 5. kids	+1.00, +0.75, +0.67, +0.54, +0.49
Ex 4 [Non-Fake]	8	+0.001376	1. browser, 2. this, 3. ram, 4. a, 5. doesn	-1.00, -0.48, -0.35, +0.20, -0.14
Ex 5 [Non-Fake]	8	-0.002151	1. this, 2. trip, 3. a, 4. da, 5. on	-1.00, -0.52, +0.49, +0.46, +0.40
Ex 6 [Non-Fake]	15	+0.017254	1. exam, 2. patiently, 3. around, 4. place, 5. for	-1.00, -0.41, -0.35, -0.30, -0.28

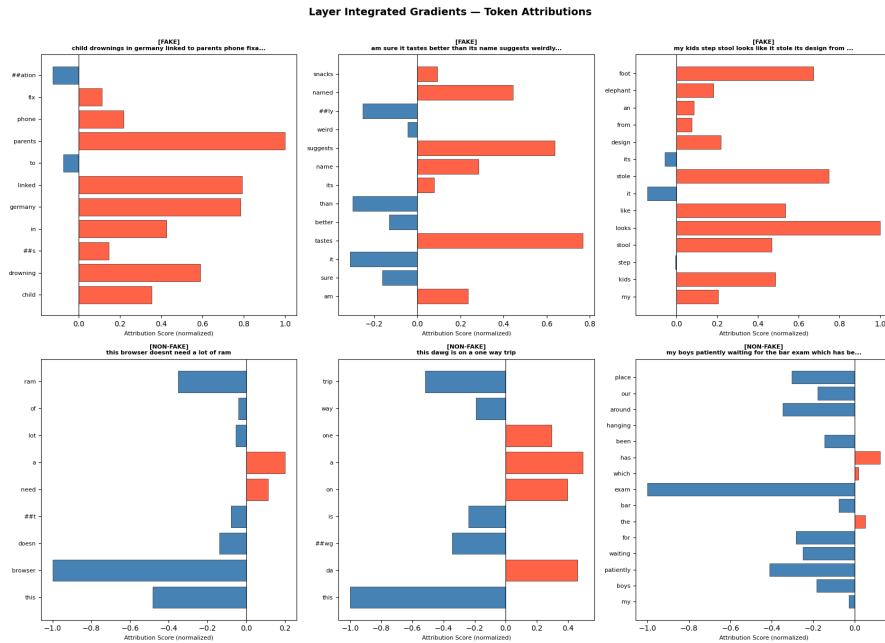


Figure 3: Token attribution profiles generated by Layer Integrated Gradients across six test examples (Green indicates positive attribution pushing toward the target class; Red indicates negative attribution pushing away).

2. **Technical and Institutional Anchors:** For non-fake instances, concrete nouns such as **browser** (-1.00), **ram** (-0.35), and **exam** (-1.00) strongly drive predictions toward genuine reporting, indicating the presence of sober, factual discourse.
3. **Contextual Polarity:** Tokens like **it** (-0.31) and **than** (-0.30) in Example 2 exhibit negative attributions, demonstrating that syntax and comparative clauses can moderate or temper the model’s confidence in a fake label.

6 Conclusion

In this project, we successfully designed and validated an encoder-only Transformer (BERT) from first principles in PyTorch (22.46M parameters), verified tensor flow across attention and feed-forward sub-layers, and applied modern transfer learning to the Fakeddit dataset. Fine-tuning **bert-base-uncased** with AdamW yielded a strong 79.10% accuracy and 83.2% recall on fake news. Finally, explainability via Captum Layer Integrated Gradients ($n_{\text{steps}} = 400$) rigorously established that lexical sensationalism drives fake news classifications, while specific technical nouns anchor non-fake determinations.

References

- [1] Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS 2017.
- [2] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL-HLT 2019.
- [3] Nakamura, K., Levy, S., & Wang, W. Y. (2020). *Fakeddit: A New Multimodal Benchmark Dataset for Fine-grained Fake News Detection*. LREC 2020.
- [4] Sundararajan, M., Taly, A., & Yan, Q. (2017). *Axiomatic Attribution for Deep Networks*. ICML 2017.
- [5] Kokhlikyan, N., et al. (2020). *Captum: A unified and generic model interpretability library for PyTorch*. arXiv:2009.07896.